



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

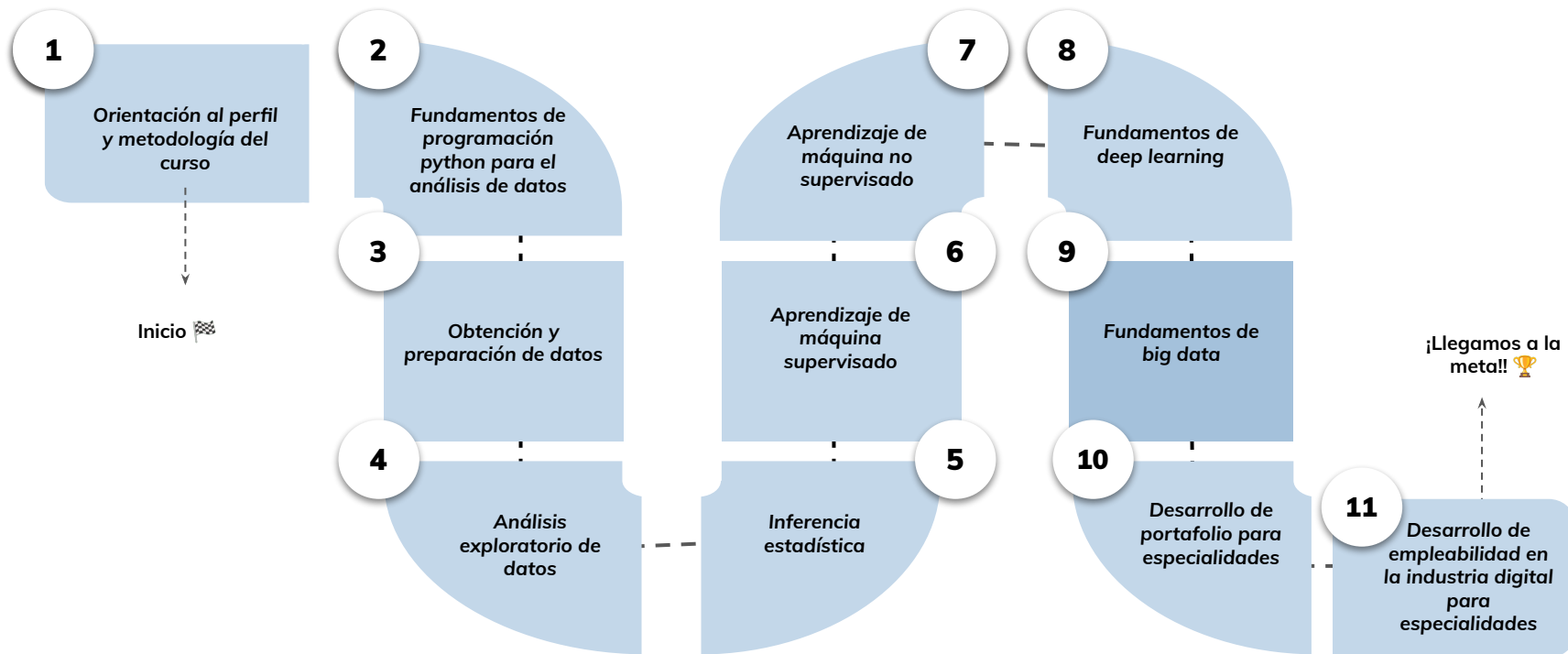


› Introducción a machine learning escalable- Parte II

Aprendizaje Esperado 5: Elaborar un modelo de machine learning (aprendizaje de máquina) utilizando mllib de apache spark para resolver un problema de grandes volúmenes de datos.

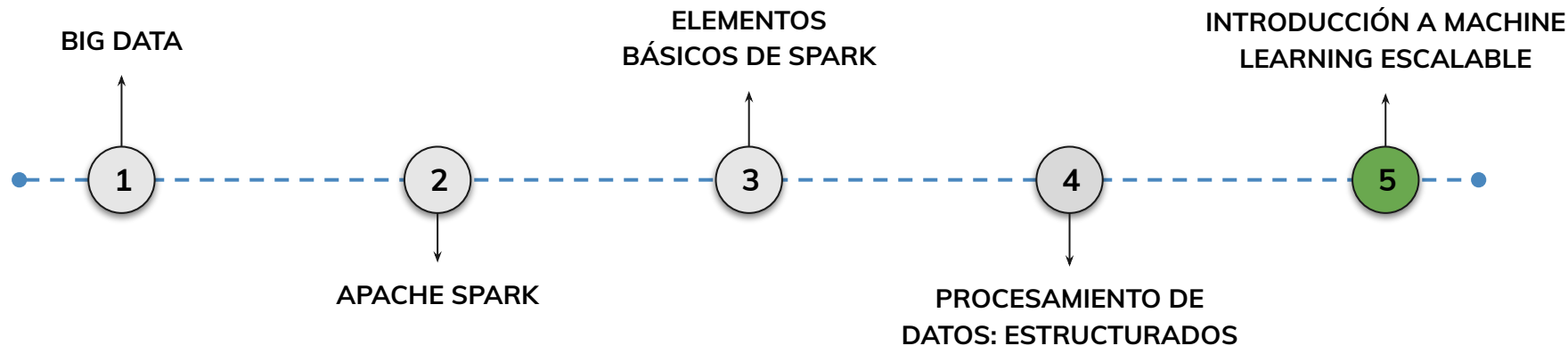
Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Implementación de
algoritmos supervisados y
no supervisados con MLlib

Implementaremos modelos de Machine Learning en Spark MLlib, aplicando algoritmos supervisados y no supervisados, desde la preparación de datos hasta la evaluación.

Implementación de
algoritmos supervisados

Flujo de trabajo: preparación, definición, entrenamiento, predicción, evaluación.

Ejemplo con regresión logística y regresión lineal.

Implementación de
algoritmos no
supervisados

Algoritmos de clustering: K-Means, Gaussian Mixture Model (GMM).

Reducción de dimensionalidad con PCA.

Objetivos de aprendizaje

¿Qué aprenderás?

- Implementar un modelo supervisado en Spark MLlib.
- Entrenar, predecir y evaluar su rendimiento en un dataset grande.
- Aplicar un algoritmo no supervisado y analizar sus resultados.
- Utilizar transformadores como VectorAssembler y StringIndexer.
- Integrar el flujo completo en un Pipeline reproducible.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Conocimos MLlib y sus características para procesamiento distribuido.
- Exploramos las estructuras de datos clave: DataFrame, vectores, matrices.
- Identificamos algoritmos supervisados y no supervisados disponibles.
- Analizamos el concepto de Pipeline para flujos de ML escalables.

Introducción a machine learning escalable

Pipeline: flujos de trabajo

Definición

Un Pipeline es una secuencia de etapas que incluyen transformadores y estimadores, permitiendo encadenar operaciones de forma coherente.

Estimadores

Componentes que aprenden a partir de los datos (por ejemplo, un modelo de regresión).

Transformadores

Componentes que transforman un DataFrame en otro (por ejemplo, escalado de características).

Ejecución

Al ejecutar un pipeline, cada etapa se aplica secuencialmente, facilitando la reproducibilidad y mantenimiento del flujo de trabajo.

Model y Transformer

Model

Representa un modelo entrenado, resultado de aplicar un estimador sobre un conjunto de datos. Contiene los parámetros aprendidos durante el entrenamiento.

Transformer

Componente que aplica transformaciones sobre nuevos datos, ya sea para preprocesamiento o para realizar predicciones con un modelo entrenado.

Estas estructuras permiten separar claramente la fase de entrenamiento de la fase de aplicación del modelo, facilitando su despliegue en entornos de producción.

Resumen de estructuras de datos

DataFrame

Para la gestión de los datos de entrada y salida, organizados en columnas y filas distribuidas.

Vector (Dense y Sparse)

Para representar las características numéricas de forma eficiente según su densidad.

Matriz

Para operaciones avanzadas que requieren representación bidimensional de datos.

Pipeline

Para la construcción de flujos de trabajo completos y reproducibles.

Model y Transformer

Para la aplicación y evaluación de modelos entrenados.

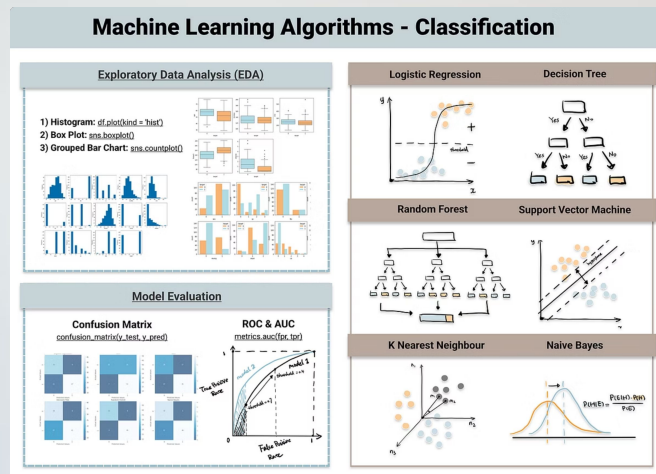
Importancia de las estructuras de datos

- ① Estas estructuras forman la base sobre la cual se construyen y ejecutan los algoritmos de Machine Learning en Spark MLlib, permitiendo un procesamiento eficiente y escalable de grandes volúmenes de datos.

La elección adecuada de las estructuras de datos es fundamental para optimizar el rendimiento y la eficiencia de los modelos de Machine Learning en entornos distribuidos.

Algoritmos de Machine Learning soportados por MLlib

MLlib ofrece un conjunto amplio de algoritmos de **aprendizaje automático** diseñados para funcionar en entornos distribuidos y adaptados a diferentes tipos de problemas. Estos algoritmos cubren tanto el aprendizaje supervisado como el no supervisado, permitiendo a los profesionales de datos resolver múltiples desafíos analíticos sobre grandes volúmenes de información.



Algoritmos de aprendizaje supervisado: Clasificación



Regresión logística

Utilizada para problemas de clasificación binaria y multiclase, estimando la probabilidad de pertenencia a cada categoría.



Árboles de decisión

Modelos basados en reglas de decisión jerárquicas que dividen los datos según valores de características.



Random Forest

Conjunto de árboles de decisión que mejora la precisión del modelo mediante la combinación de múltiples clasificadores.



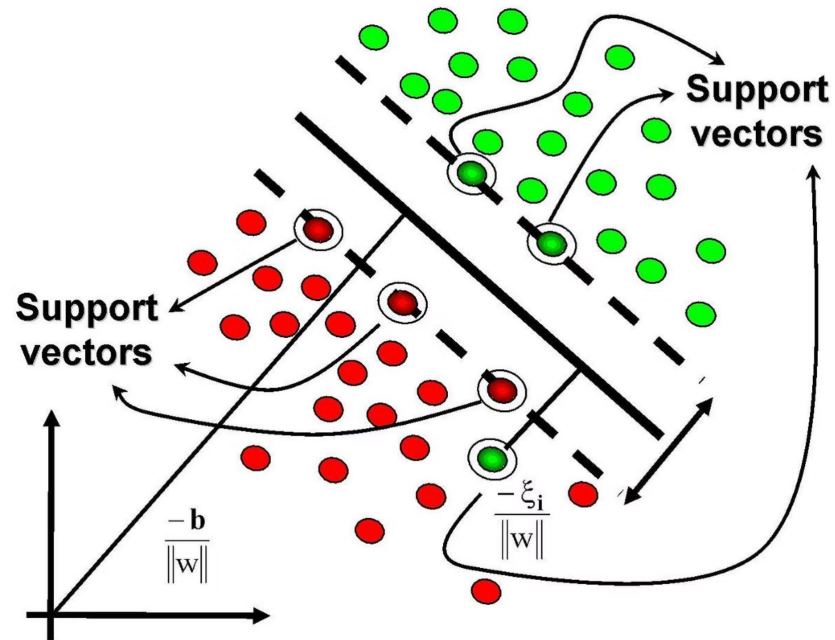
Gradient-Boosted Trees

Técnica basada en boosting para mejorar el rendimiento predictivo mediante la construcción secuencial de modelos.

Algoritmos de aprendizaje supervisado: Clasificación (continuación)

Support Vector Machines (SVM)

Algoritmo para clasificación binaria que busca el hiperplano que mejor separa las clases con el margen máximo.



Naive Bayes

Clasificador probabilístico basado en el teorema de Bayes, eficiente para problemas con muchas características.

Variantes implementadas:

- Multinomial Naive Bayes
- Bernoulli Naive Bayes
- Gaussian Naive Bayes

Algoritmos de aprendizaje supervisado: Regresión

Regresión lineal

Para problemas de predicción de valores continuos mediante la estimación de una función lineal.

Regresión con árboles de decisión

Utiliza árboles para predecir valores numéricos, capturando relaciones no lineales.

Random Forest Regressor

Conjunto de árboles de regresión que mejora la precisión y reduce el sobreajuste.

Gradient-Boosted Trees Regressor

Técnica de boosting aplicada a problemas de regresión para mejorar el rendimiento predictivo.

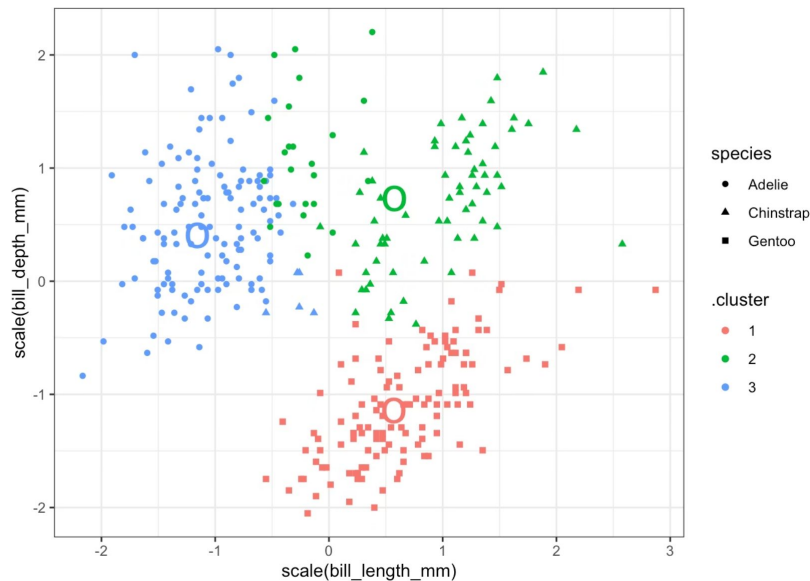
Algoritmos de aprendizaje no supervisado: Clustering

K-Means

Algoritmo para el agrupamiento de datos en un número predefinido de clústeres basándose en la similitud entre las observaciones.

Características:

- Escalable a grandes conjuntos de datos
- Implementación paralela eficiente
- Inicialización inteligente (K-means++)



Algoritmos de aprendizaje no supervisado: Clustering (continuación)



Gaussian Mixture Model (GMM)

Técnica probabilística para clustering basada en distribuciones gaussianas, útil cuando los clústeres tienen diferentes formas y tamaños.



Bisecting K-Means

Variante jerárquica del algoritmo K-Means que divide iterativamente los clústeres existentes.



Power Iteration Clustering

Algoritmo de clustering basado en la propagación de similitudes a través de un grafo de datos.

Algoritmos de aprendizaje no supervisado: Reducción de dimensionalidad

Principal Component Analysis (PCA)

Técnica para la reducción de la dimensionalidad que transforma los datos a un espacio de menor dimensión, conservando la mayor cantidad de información posible.

Beneficios:

- Reducción del ruido en los datos
- Visualización de datos multidimensionales
- Mejora del rendimiento de algoritmos posteriores

```
from pyspark.ml.feature import PCA

pca = PCA(k=2, inputCol="features", outputCol="pcaFeatures")
modelo_pca = pca.fit(data)
resultado = modelo_pca.transform(data)
resultado.select("pcaFeatures").show()
```

Algoritmos de aprendizaje no supervisado: Reducción de dimensionalidad (continuación)

Singular Value Decomposition (SVD)

Técnica de factorización matricial que descompone una matriz en tres componentes, útil para compresión de datos y eliminación de ruido.

Latent Dirichlet Allocation (LDA)

Modelo probabilístico generativo utilizado para la identificación de temas en colecciones de texto, permitiendo descubrir patrones latentes en documentos sin necesidad de supervisión.

Algoritmos de filtrado colaborativo

MLlib incluye algoritmos de **filtrado colaborativo**, utilizados principalmente en sistemas de recomendación. Uno de los más destacados es:



Alternating Least Squares (ALS): Empleado para la predicción de preferencias de usuarios en plataformas como e-commerce, música o contenido audiovisual. Permite construir recomendaciones personalizadas basadas en el comportamiento pasado de usuarios similares.

Optimización para entornos distribuidos

Todos estos algoritmos han sido optimizados para funcionar sobre clústeres distribuidos, utilizando las estructuras de datos de Spark como **DataFrames** y **Vectores**. Esto permite que las tareas de entrenamiento, predicción y evaluación se realicen de forma eficiente incluso con millones de registros.

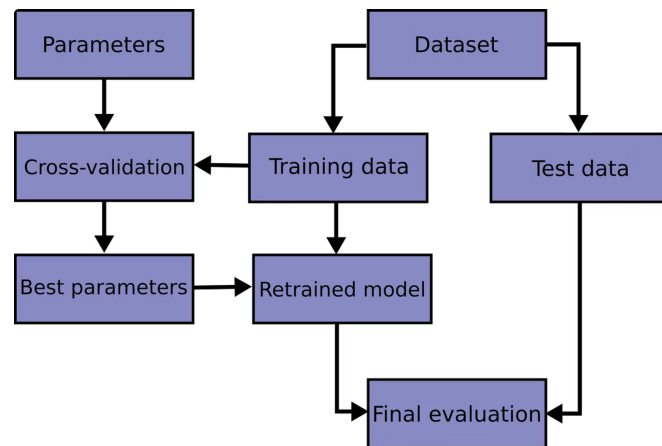
Soporte para pipelines y validación

Pipelines

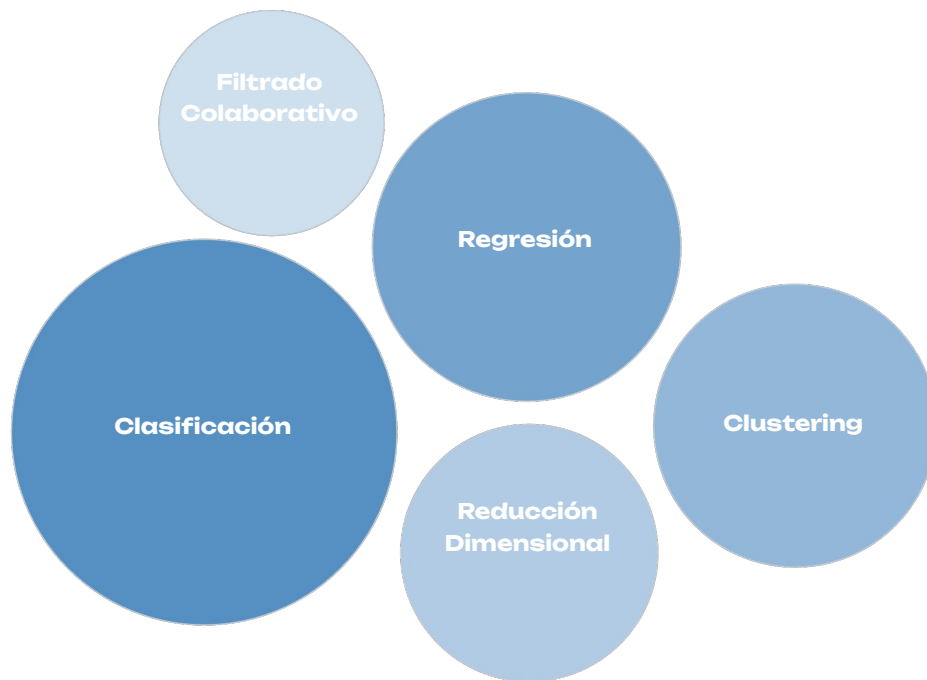
La mayoría de los algoritmos soportan la creación de **pipelines**, permitiendo encadenar las etapas de preparación de datos, entrenamiento y evaluación en un flujo automatizado.

Validación y optimización

MLlib ofrece herramientas para la **validación de modelos** y la **búsqueda de hiperparámetros**, facilitando la optimización de los algoritmos mediante técnicas como la **validación cruzada** y el **grid search**.



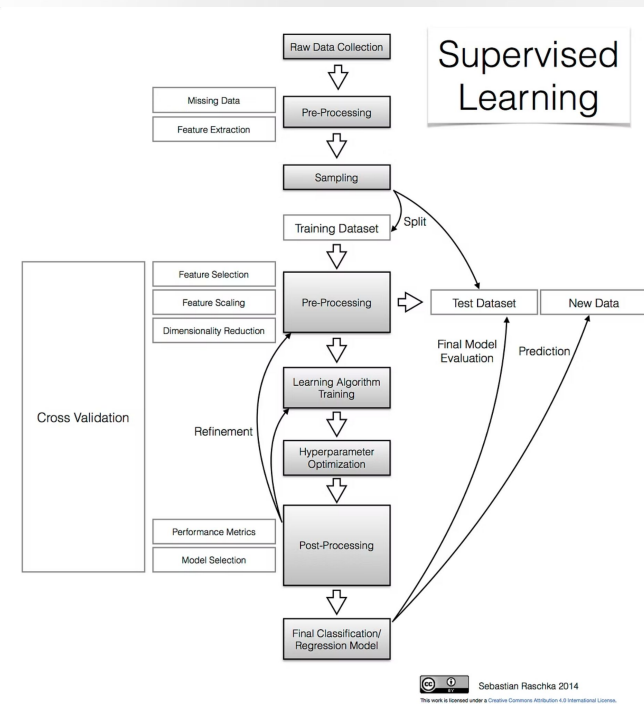
Resumen de algoritmos



En resumen, los algoritmos soportados por MLlib permiten abordar una amplia gama de problemas de Machine Learning de manera escalable, eficiente y completamente integrada con el ecosistema de Apache Spark.

Implementación de algoritmos de Machine Learning supervisados con MLlib

El uso de **MLlib** para la implementación de algoritmos supervisados permite construir modelos predictivos capaces de aprender a partir de datos etiquetados. Los problemas supervisados más comunes son la **clasificación** y la **regresión**.



Pasos para implementar algoritmos supervisados

1

Preparación de datos

Organización de los datos en un DataFrame con columnas para etiquetas y características.

2

Definición del algoritmo

Selección e instanciación del modelo adecuado para el problema.

3

División de datos

Separación en conjuntos de entrenamiento y prueba.

4

Entrenamiento

Ajuste del modelo con los datos de entrenamiento.

5

Predicción

Aplicación del modelo entrenado a nuevos datos.

6

Evaluación

Medición del rendimiento del modelo con métricas adecuadas.

Preparación de los datos

Para llevar a cabo la implementación de un algoritmo supervisado con MLlib, el primer paso es la **preparación de los datos**. Los datos deben organizarse en un **DataFrame** que contenga una columna con la etiqueta (label) y otra con un vector de características (features).

VectorAssembler

Transformador que combina múltiples columnas en un único vector de características.

StringIndexer

Convierte columnas de texto en índices numéricos, necesario para variables categóricas.

OneHotEncoder

Transforma variables categóricas en vectores binarios para su uso en algoritmos.

Normalizer/StandardScaler

Normaliza o estandariza las características numéricas para mejorar el rendimiento.

Definición del algoritmo

Para clasificación binaria

```
from pyspark.ml.classification import  
LogisticRegression# Crear el modelolr =  
LogisticRegression(    maxIter=10,    regParam=0.01,  
elasticNetParam=0.8,    labelCol="label",  
featuresCol="features")
```

Para problemas de regresión

```
from pyspark.ml.regression import LinearRegression#  
Crear el modelolr = LinearRegression( maxIter=10,  
regParam=0.3, elasticNetParam=0.8, labelCol="label",  
featuresCol="features")
```

División de los datos

El siguiente paso consiste en **dividir los datos en conjuntos de entrenamiento y prueba** para evaluar el desempeño del modelo:

```
# Dividir los datos en conjuntos de entrenamiento (70%) y prueba (30%)
train_data, test_data = data.randomSplit([0.7, 0.3], seed=42)
print(f"Datos de entrenamiento: {train_data.count()} registros")
print(f"Datos de prueba: {test_data.count()} registros")
```

Esta división permite entrenar el modelo con un subconjunto de datos y evaluarlo con datos que no ha visto durante el entrenamiento, obteniendo así una medida más realista de su capacidad de generalización.

Entrenamiento del modelo

A continuación, se **entrena el modelo** utilizando el método `.fit()` sobre el conjunto de entrenamiento:

```
# Entrenar el modelomodel = lr.fit(train_data)# Mostrar los coeficientes y el intercepto (para  
regresión)print(f"Coeficientes: {model.coefficients}")print(f"Intercepto: {model.intercept}")
```

Durante esta fase, el algoritmo ajusta sus parámetros internos para minimizar el error de predicción en los datos de entrenamiento.

Realización de predicciones

Una vez entrenado, se pueden realizar **predicciones** sobre el conjunto de prueba:

```
# Realizar predicciones
predictions = model.transform(test_data)
# Mostrar algunas predicciones
predictions.select("features", "label", "prediction").show(5)
```

El método `transform()` aplica el modelo entrenado a nuevos datos, generando una columna adicional con las predicciones.

Evaluación del modelo

Para clasificación binaria

```
from pyspark.ml.evaluation import
BinaryClassificationEvaluator# Evaluar el
modeloevaluator = BinaryClassificationEvaluator(
labelCol="label",
rawPredictionCol="rawPrediction",
metricName="areaUnderROC")auc =
evaluator.evaluate(predictions)print(f"Área bajo la
curva ROC: {auc}")
```

Para problemas de regresión

```
from pyspark.ml.evaluation import
RegressionEvaluator# Evaluar el modeloevaluator =
RegressionEvaluator( labelCol="label",
predictionCol="prediction", metricName="rmse")rmse =
evaluator.evaluate(predictions)print(f"Error
cuadrático medio: {rmse}")
```


Uso de Pipelines

Una práctica recomendada es integrar todas estas etapas en un **Pipeline**, lo que permite automatizar el flujo de trabajo y facilita la reutilización y mantenimiento del modelo.

```
from pyspark.ml import Pipeline# Crear el pipelinepipeline = Pipeline(stages=[ vectorAssembler, # Transformador
para crear el vector de características scaler, # Opcional: normalización de características lr # Algoritmo de
aprendizaje])# Entrenar el pipeline completopipeline_model = pipeline.fit(train_data)# Aplicar el pipeline a los
datos de pruebapredictions = pipeline_model.transform(test_data)
```

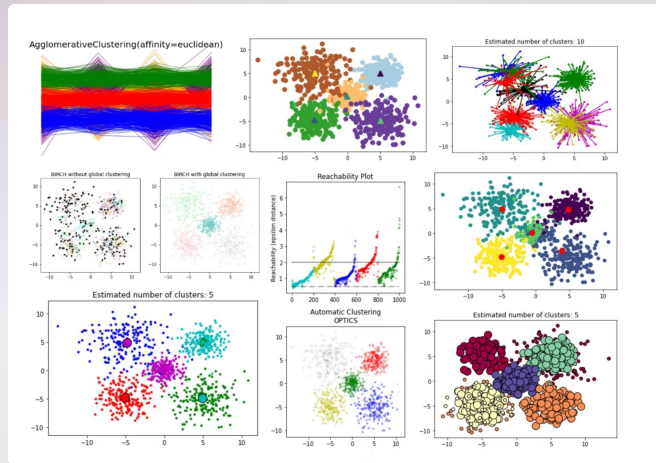
Resumen de implementación supervisada



La implementación de algoritmos supervisados con MLlib es escalable y puede aplicarse sobre grandes conjuntos de datos distribuidos, lo que la convierte en una solución ideal para entornos de Big Data.

Implementación de algoritmos de Machine Learning no supervisados con MLlib

Además de los algoritmos supervisados, **MLlib** proporciona un conjunto de herramientas para implementar algoritmos de **aprendizaje no supervisado**. Estos algoritmos permiten identificar patrones, estructuras y relaciones ocultas en los datos sin necesidad de contar con etiquetas o valores objetivo.



Implementación de K-Means

Uno de los algoritmos no supervisados más utilizados es el **K-Means**, que permite agrupar datos en un número predefinido de clústeres, basándose en la similitud entre las observaciones. La implementación de K-Means con MLlib sigue un flujo similar al de los algoritmos

```
from pyspark.ml.clustering import KMeans# Crear una instancia del algoritmo K-Meanskmeans = KMeans(k=3, # Número de clústeres seed=42, # Semilla para reproducibilidad featuresCol="features", predictionCol="cluster")# Entrenar el modelomodel = kmeans.fit(data)# Obtener los centroides de los clústerescenters = model.clusterCenters()print("Centroides de los clústeres:")for center in centers:    print(center)
```

Predicción de clústeres y evaluación

Luego, se **entrena el modelo** utilizando el método `.fit()`:

```
# Predecir a qué clúster pertenece cada
observación predictions = model.transform(data) #
Mostrar algunas
predicciones predictions.select("features",
"cluster").show(5)
```

MLlib también ofrece herramientas para **evaluar la calidad del agrupamiento**, como el **Silhouette Score**, que mide qué tan bien definidos están los clústeres:

```
from pyspark.ml.evaluation import
ClusteringEvaluator # Evaluar el modelo evaluator =
ClusteringEvaluator( predictionCol="cluster",
featuresCol="features",
metricName="silhouette") silhouette =
evaluator.evaluate(predictions) print(f"Silhouette
Score: {silhouette}")
```

Gaussian Mixture Model (GMM)

Otro algoritmo no supervisado disponible en MLlib es el **Gaussian Mixture Model (GMM)**, que utiliza una combinación de distribuciones gaussianas para modelar los datos. Este enfoque es útil cuando los clústeres tienen diferentes formas y tamaños.

```
from pyspark.ml.clustering import GaussianMixture# Crear una instancia del algoritmo GMMgmm =  
GaussianMixture( k=3, # Número de componentes gaussianas seed=42, # Semilla para reproducibilidad  
featuresCol="features", predictionCol="cluster")# Entrenar el modelomodel = gmm.fit(data)# Mostrar los  
pesos y parámetros de cada componente  
for i in range(model.getK()): print(f"Peso de la componente {i}:  
{model.weights[i]}") print(f"Media de la componente {i}: {model.gaussiansDF.collect()[i].mean}")  
print(f"Covarianza de la componente {i}: {model.gaussiansDF.collect()[i].cov}")
```

Reducción de dimensionalidad con PCA

Además, MLlib soporta técnicas de **reducción de dimensionalidad** como el **Análisis de Componentes Principales (PCA)**.

PCA permite transformar los datos a un espacio de menor dimensión, conservando la mayor cantidad de información posible:

```
from pyspark.ml.feature import PCA# Crear una instancia de PCApca = PCA( k=2, # Número de componentes
principales inputCol="features", outputCol="pcaFeatures")# Ajustar el modelo PCAmoel = pca.fit(data)#
Transformar los datos al espacio de componentes principalesresult = moel.transform(data)# Mostrar los
resultadosresult.select("pcaFeatures").show(5)# Obtener la matriz de explicación de
varianzaexplained_variance = moel.explainedVarianceprint(f"Varianza explicada: {explained_variance}")
```

Resumen de implementación no supervisada

Estas herramientas permiten a los analistas y científicos de datos descubrir información valiosa en conjuntos de datos masivos, facilitando la toma de decisiones basada en patrones ocultos.

Live Coding

¿En qué consistirá la Demo?

Implementaremos paso a paso un modelo supervisado y uno no supervisado en Spark MLlib sobre un dataset simulado de gran tamaño.

1. Preparar datos y convertir variables con `StringIndexer`, `OneHotEncoder` y `VectorAssembler`.
2. Entrenar un modelo supervisado (ej. `LogisticRegression` para clasificación).
3. Evaluar el modelo con métricas como Área bajo la curva ROC o RMSE.
4. Implementar un modelo no supervisado (ej. `K-Means`) y evaluar con `Silhouette Score`.



Momento:

Time-out!



5 -10 min.





Ejercicio

Construye y evalúa tu modelo MLlib

Construye y evalúa tu modelo MLlib

Contexto: 🙌

Aplicarás MLlib para construir y evaluar un modelo supervisado y uno no supervisado usando un dataset de gran tamaño.

Consigna: 📝

1. Selecciona un dataset con variables categóricas y numéricas.
2. Prepara los datos aplicando transformadores adecuados.
3. Implementa un modelo supervisado (clasificación o regresión) y evalúalo.
4. Implementa un modelo no supervisado (K-Means o PCA) y analiza sus resultados.
5. Integra todo en un Pipeline y documenta el flujo.

Paso a paso: ⚙️

1. Cargar datos y realizar preprocesamiento.
2. Dividir datos en entrenamiento y prueba.
3. Entrenar ambos modelos y evaluar métricas.
4. Comparar resultados y optimizar parámetros.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Implementamos modelos supervisados en Spark MLlib.
- ✓ Entrenamos, predecimos y evaluamos modelos con grandes volúmenes de datos.
- ✓ Aplicamos K-Means y PCA para clustering y reducción de dimensionalidad.
- ✓ Usamos VectorAssembler y otros transformadores para preparar datos.
- ✓ Construimos Pipelines para flujos reproducibles de ML escalable.



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

